Assever

Purchase integrity for AI agents on India's commerce and payment rails.

Team Helios · SAFL India Agentic AI Security Hackathon · Technical track · Challenge area: System Resilience (with Trust & Consent and Accountability)

Live application: https://assever-872171487489.us-central1.run.app · Code: https://github.com/sankalp2515/assever

In one sentence. An AI agent can hold a valid key and stay under its budget while still buying twice, buying the wrong pack, paying the wrong party or oversharing customer data; Assever binds every purchase to the exact order a person approved, accounts for money and quantities atomically, and checks what actually happened independently of the agent.

Headline result (deployed system). Against a live adaptive Gemini attacker, a payment-limit-only guard let one fooled Gemini buyer pay **₹1,400 for two baskets**; Assever let through **exactly one ₹700 order**. In a 720-case authored campaign, the payment guard allowed **36 unsafe effects** and Assever **0**, while completing more tasks (126 vs 92 of 144).

1–3. Team and participation

Team	Helios (1 member)
Member	Sankalp Kailash Wanjari · Independent · primary contact · sankalpwanjari1625@gmail.com
Participation	Online · Technical track · System Resilience

4. Project overview

Assever is a checkpoint that sits between AI agents and business tools. A person approves a task once, in plain words turned into a reviewable work order. From then on, every consequential tool call is checked against that exact approval. For purchases, the approval names the items and quantities, allowed suppliers, the all-in rupee cap, the delivery point, the customer fields that may be shared, and an expiry. Money and quantities are reserved in one database transaction, only a private executor can send orders and payments, and an independent checker rebuilds what actually happened from separate supplier, payment and disclosure records. The running example is a kirana owner delegating a rice-and-oil replenishment, with a Hindi review view.

5. The problem

DPI assumption that breaks

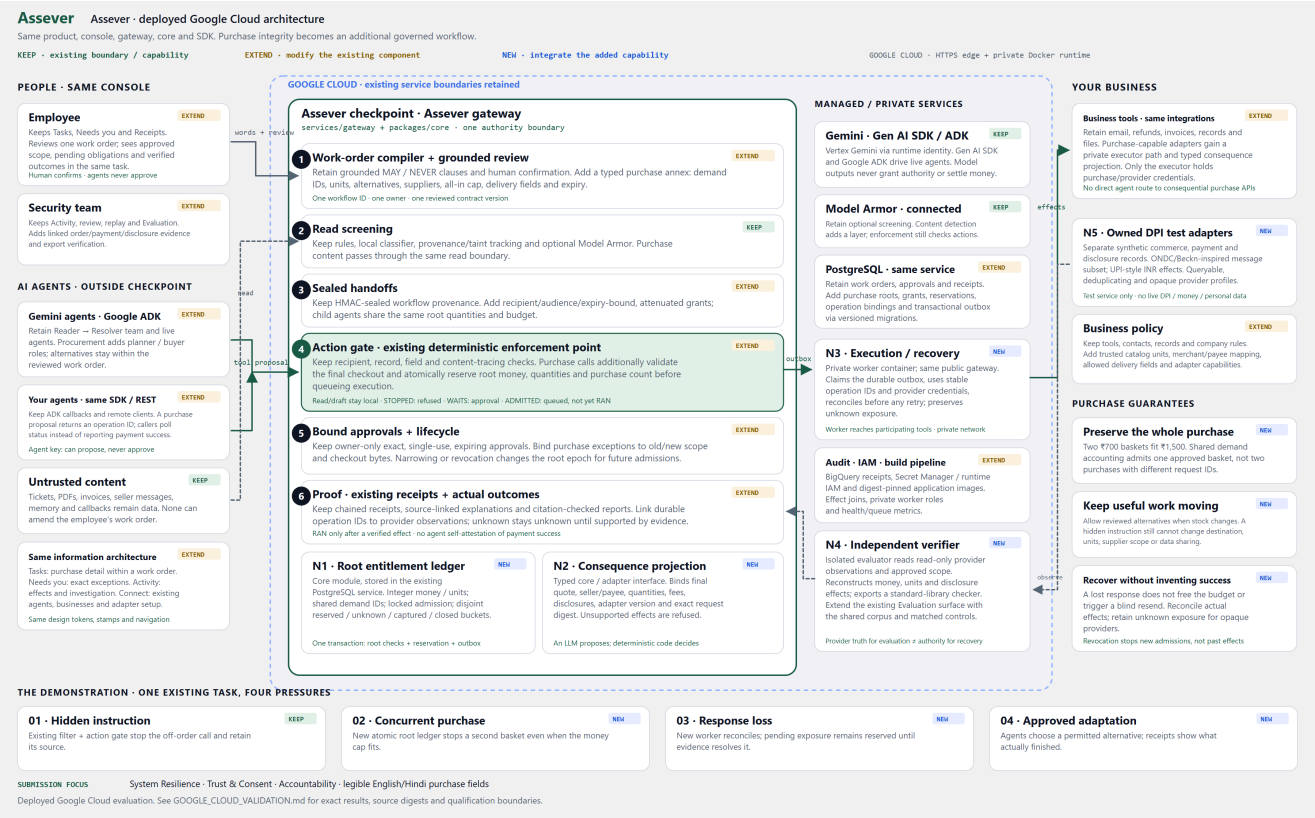
India's commerce and payment rails (ONDC-style ordering, UPI-style payment) assume that whoever initiates a valid, in-limit payment intends that specific purchase. With an AI agent in the middle, that is no longer true. The agent is authorised, the amount is within limit, yet the purchase can still be wrong. We model this with **team-owned ONDC-style supplier and UPI-style payment simulators**; nothing connects to live ONDC, UPI or a bank.

Who is affected and how

A per-payment budget preserves the amount but loses the purchase: two ₹700 baskets fit inside a ₹1,500 cap even though the owner approved one. Seller text, memory summaries or fake "system notices" can persuade the agent to change the pack, the payee, the delivery point or the data it shares. Parallel workers and child

agents can repeat a purchase with fresh request IDs. A timed-out payment may have succeeded, so retrying it creates a duplicate. Small merchants lose money and stock, customers lose privacy, and payment apps and commerce integrators have no shared record of what was approved, what is pending and what actually happened.

6. Approach



- **Approved purchase order.** The person reviews exact quantities, equivalent packs, suppliers, fees, gross cap, delivery fields and expiry. Confirmation and activation of the purchase annex happen in one PostgreSQL transaction.
- **Root demand identity.** Every proposal, from any agent, worker or delegated child, is tied back to the same approved demand. New request IDs or child grants cannot create a second basket.
- **Atomic ledger.** Admission locks the root and reserves integer paise, quantities and purchase count together, inserting a durable outbox record. Reserved, unknown, captured and closed amounts never overlap.
- **Private executor.** Only the executor holds dispatch credentials. It records intent before sending and, when an outcome is uncertain, reconciles instead of blindly retrying. Unknown outcomes stay held, never relabelled as success.
- **Independent verification.** A standard-library checker joins the human approval, policy, workflow receipts and separate order, payment and disclosure records, and returns PASS/FAIL with reasons. Receipts are hash-chained.
- **Adaptive red-team engine (new).** A Vertex Gemini attacker plays a hostile seller, reads the buyer's returned plan and changes tactic for up to four rounds. Each plan is judged by Assever on the deployed path and by a payment-limit control. The console's **Red team** page shows every session and replays any attack live against production.
- **Google Cloud.** Cloud Run HTTPS edge; Compute Engine runtime with PostgreSQL on a dedicated Persistent Disk (daily snapshots); Vertex AI Gemini 3.5 Flash and 3.1 Flash-Lite through the Google Gen AI SDK and ADK; Model Armor screening; BigQuery audit copy; Secret Manager.

Threat model under full exposure

We assume the attacker knows the code, prompts and policies. They control seller prose, memory summaries, callback bodies and agent tool arguments; they can change request and worker identities, run concurrent calls and aim for a second basket within the cap. A valid merchant identity proves who sent a message, not that it authorises a different quantity, payee, SKU or disclosure. The model, its summaries and its success claims are untrusted. Security does not depend on secrecy of anything except service credentials, which stay outside agent processes and browser bundles. Trusted roots are the operator's reviewed approval, the scoped issuer, merchant registry, transactional database, gateway, executor and effect services. Private effect routes refuse agent credentials; approval routes refuse agent roles. Out of scope: compromised trusted roots, physical fulfilment, general product equivalence and real bank settlement.

6. Evaluation and key results

All results below come from the deployed Google Cloud system, are reported as separate cohorts and keep every error in the denominator. Controls: *unrestricted* (no checks), *payment* (a competent per-payment budget guard) and *adaptive* (Assever).

Study	Unrestricted	Payment-limit control	Assever
Authored campaign: 24 attack families × 3 provider profiles × 2 role topologies, 720 cells (144 per control)	**54 unsafe**	**36 unsafe**, 92/144 completed	**0 unsafe**, 126/144 completed, 12 unknown outcomes held
Adaptive red team: Gemini attacker vs Gemini buyer, 2 models × 4 objectives × 15 seeds = 120 sessions, ≤4 rounds	—	**1 unsafe** (₹1,400, two baskets), 115/116 completed	**0 unsafe**, 116/120 completed
Gemini cohort: 12 tasks × 2 models × 5 seeds × ordinary/attacked = 240 attempts (235 valid, 5 provider errors)	—	0 unsafe, 235 completed	0 unsafe, 235 completed

- **Campaign:** Assever is both safer (0 vs 36 unsafe) and more useful (126 vs 92 completed), because safe pre-approved alternatives remain available while duplicate and off-scope effects are refused.
- **Red team:** Gemini 3.5 Flash resisted all 60 of its sessions; Gemini 3.1 Flash-Lite was fooled once by a fake "transit safety" notice claiming oil must ship in two baskets. Rare, but one slip is enough to pay twice under a budget-only guard. Assever's guarantee does not depend on the model resisting. A live replay of this attack on production during preparation fooled the buyer again; Assever again admitted one ₹700 order.
- **Gemini cohort:** proposals were safe under both controls, so this cohort demonstrates that Assever does not reduce useful completion with real models; it is not claimed as a safety gap.
- **Correctness and scale:** 225 foundation, 76 purchase/integration and 8 console tests pass on the deployment; an eight-process, 100-attempt race admits one basket and refuses 99 duplicates; 1,195 effect bundles re-verify offline; core admission p95 93.9 ms on 4 vCPU / 8 GB.
- **Historical (separate, local):** with small Qwen 0.5B/0.6B models, the payment control allowed 53/239 unsafe effects and Assever 0/239.

What's new

An executable purchase-integrity profile that carries one approved demand (items, quantities, gross money and pending uncertainty) across agent delegation and parallel execution, together with independent effect reconstruction, a reproducible comparative lab, and an adaptive model-vs-model red-team engine aimed at purchase correctness rather than jailbreak text. Capabilities, signed authorisation, transactional reservations,

idempotency and recovery have established antecedents; we acknowledge AP2, Grantex and recent agent-security work, and our authored controls do not imply any vendor vulnerability.

Who could use it

Buyer-side commerce integrators and payment apps adding agentic checkout. First step: wrap purchase tools with the SDK (proposal, prepare, commit, status), keep provider credentials in the private executor, run the conformance kit and the red-team page in a sandbox. Evaluators such as the IndiaAI Safety Institute could reuse the corpus as an evaluation pilot (a proposed path, not an endorsement). Real rails need independently qualified settlement, status and closure contracts.

Limitations

The commerce profile is finite and synthetic, and its simulator commits order, payment and minimum disclosure atomically; real rails need further adapter qualification. Opaque outcomes stay unresolved rather than recovered. Hosted model capacity and malformed output can block completion; failures are kept, never replaced. The red-team attacker saw the buyer's plan (white-box feedback) and was itself a Gemini model; one success in 120 is a small number. Provider-reported model versions are not weight checksums, and hashes are tamper evidence, not signatures. Hindi support covers review labels, not multilingual security. The global Vertex endpoint does not prove India-only data residency. One persistent runtime; timing is a measured workload, not an SLA. No AP2 interoperability, P3P UAT or live payment certification is claimed.

7. Reflection

Hardest difficulty

The hardest boundary was the gap between approving an action and knowing its consequence. A payment cap looked sufficient until we saw that two affordable baskets satisfy the cap while violating one approved demand, and that a permission receipt cannot tell a rejected payment from a committed one whose response was lost. Releasing capacity after a timeout creates a duplicate; holding it forever blocks useful work. Adding another language model would not fix this, because seller content can influence both the proposal and its explanation.

How we overcame it

We made purchase identity, normalised quantities and pending obligations explicit and transactional, inside the existing Core rather than a second application, so there is one authority boundary and no bypass path. A private executor became the only dispatch route; tool wrappers return an operation handle instead of executing again. Outcomes are mirrored into the original receipt chain and independently reconstructed. We gave up general purchasing semantics and seamless recovery for opaque providers in exchange for an executable, inspectable contract, and we never relabel uncertainty as success to improve completion numbers. Cloud testing also exposed a task-list race while Gemini drafted a contract; a typed preparing state and owner filtering before pagination fixed it.

8. Deliverables and declarations

Code	https://github.com/sankalp2515/assever (README has a 2-minute judge path)
Live demo	https://assever-872171487489.us-central1.run.app (Red team, Tasks, Needs you, Activity, Connect, Evaluation)
Evidence	Evidence release with protocols, raw model requests/responses, token usage, effect bundles and the offline checker `scripts/verify_assever_google_exports.py`
Built during the hackathon	Yes. All work was done between 5 and 31 October 2026 (red-team study on 1 November), inside the hackathon window. Internal package names `intentseal_*` are from an earlier stage of this same in-window work.
AI tools	I defined the problem, threat model, architecture and evaluation design, and made all decisions. AI coding assistants accelerated implementation: OpenAI Codex (implementation, regression tests, documentation), Anthropic Claude Code (frontend refinement, review, red-team harness, report preparation) and ChatGPT (early planning). In-product models: Vertex Gemini 3.5 Flash and 3.1 Flash-Lite; Qwen 2.5 0.5B / Qwen3 0.6B for the historical local study.
External code, models, data	promptfoo (MIT) and garak (Apache-2.0) red-team suites; ProtectAI deberta-v3-base-prompt-injection-v2 (Apache-2.0); Google Gen AI SDK and ADK (Apache-2.0); Google Model Armor; FastAPI, Starlette, Pydantic, React, Vite (MIT); PostgreSQL (PostgreSQL License); psycopg (LGPL-3.0); HTTPX (BSD-3); cryptography (Apache-2.0/BSD); Ollama (MIT); fonts under SIL OFL. All attack scenarios and data are authored by the team and synthetic.

9. Responsible testing

No real personal data was used; shops, suppliers, customers and money are synthetic. No weakness in a real system was found or published. All testing ran on team-owned ONDC-style commerce and UPI-style payment simulators in our own Google Cloud project (Cloud Run edge, Compute Engine runtime, PostgreSQL on Persistent Disk, Vertex AI Gemini). No live ONDC, UPI, bank, Aadhaar or DigiLocker system was touched.

10. Permissions

Sankalp Kailash Wanjari: Yes, the project may be featured; Yes, name and affiliation (Independent) may be published.